

Experiment No.	3
Experiment Title	Regularized Linear Regression Models on the Loan Prediction Dataset ¹
Student Name	Naren Kandasamy
Register Number	3122247001036
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

Anaconda Activation Command:

```
source /opt/anaconda3/bin/activate anaconda-navigation
```

1 Objective

This experiment looks at loan amount prediction using plain OLS Linear Regression next to three regularized versions – Ridge, Lasso, and ElasticNet – with the main interest being how each penalty term changes the coefficients and whether that translates into better performance on data the model hasn't seen.

2 Problem Statement

Figuring out a reasonable loan amount from an applicant's details is a pretty standard regression problem in finance. The tricky part isn't fitting a line – it's working out which of the applicant features (income, credit history, and so on) actually matter, and making sure the model doesn't just memorize the training set. That second part is where regularization comes in.

3 Dataset Description

Dataset Name	Loan Prediction Dataset
Dataset Source	Kaggle
Number of Samples	~614
Number of Features	12
Number of Classes	N/A (Regression task)
Missing Values	Imputed (Median for numeric, Mode for categorical)
Train-Test Split	80% Train, 20% Test

4 Exploratory Data Analysis

As before, an automated EDA function was used to put together a 12-subplot summary of the feature distributions and how the variables relate to one another.

5 Data Preprocessing

There was a fair bit to do here before the data was ready for a linear model:

- **Imputation:** numeric gaps (LoanAmount, Loan_Amount_Term, Credit_History) were filled with the median; categorical gaps went with the mode instead.
- **Encoding:** the categorical string columns were label-encoded so everything is numeric.
- **Scaling:** `StandardScaler` was applied across the board. This one's not optional for Ridge or Lasso – if features aren't on the same scale, the penalty ends up hitting some coefficients harder than others just because of how the original units happened to be measured, not because those features are actually less important.
- **Train-Test Split:** 80:20.

6 Machine Learning Algorithm

Four regression variants were tried out:

- **Linear Regression (OLS):** the standard approach – minimize the sum of squared residuals, no penalty involved.
- **Ridge Regression (L2):** adds a penalty on the squared size of the coefficients, controlled by α . Coefficients shrink, but generally all move toward zero together rather than any single one dropping out.
- **Lasso Regression (L1):** similar idea, but the penalty is on the absolute value instead. This has the interesting side effect that some coefficients get pushed exactly to zero, which is basically automatic feature selection.
- **ElasticNet:** a mix of both penalties, trying to get some of Ridge's stability along with some of Lasso's tendency to zero out weak features.

7 Experimental Setup

Python Version	3.10+
Scikit-Learn Version	1.0+
IDE	VS Code
Operating System	Linux
Hardware	Standard CPU Workspace

8 Hyperparameter Selection

Parameter	Value
Ridge 'alpha'	10.0
Lasso 'alpha'	1.0
ElasticNet 'alpha'	1.0
ElasticNet 'l1_ratio'	0.5

9 Results

Metric	Value
Linear Reg. MSE	1928.32
Linear Reg. R^2	0.2872
Ridge MSE	1918.45
Ridge R^2	0.2908
Lasso MSE	1926.88
Lasso R^2	0.2877
ElasticNet MSE	2012.01
ElasticNet R^2	0.2562

10 Result Visualization

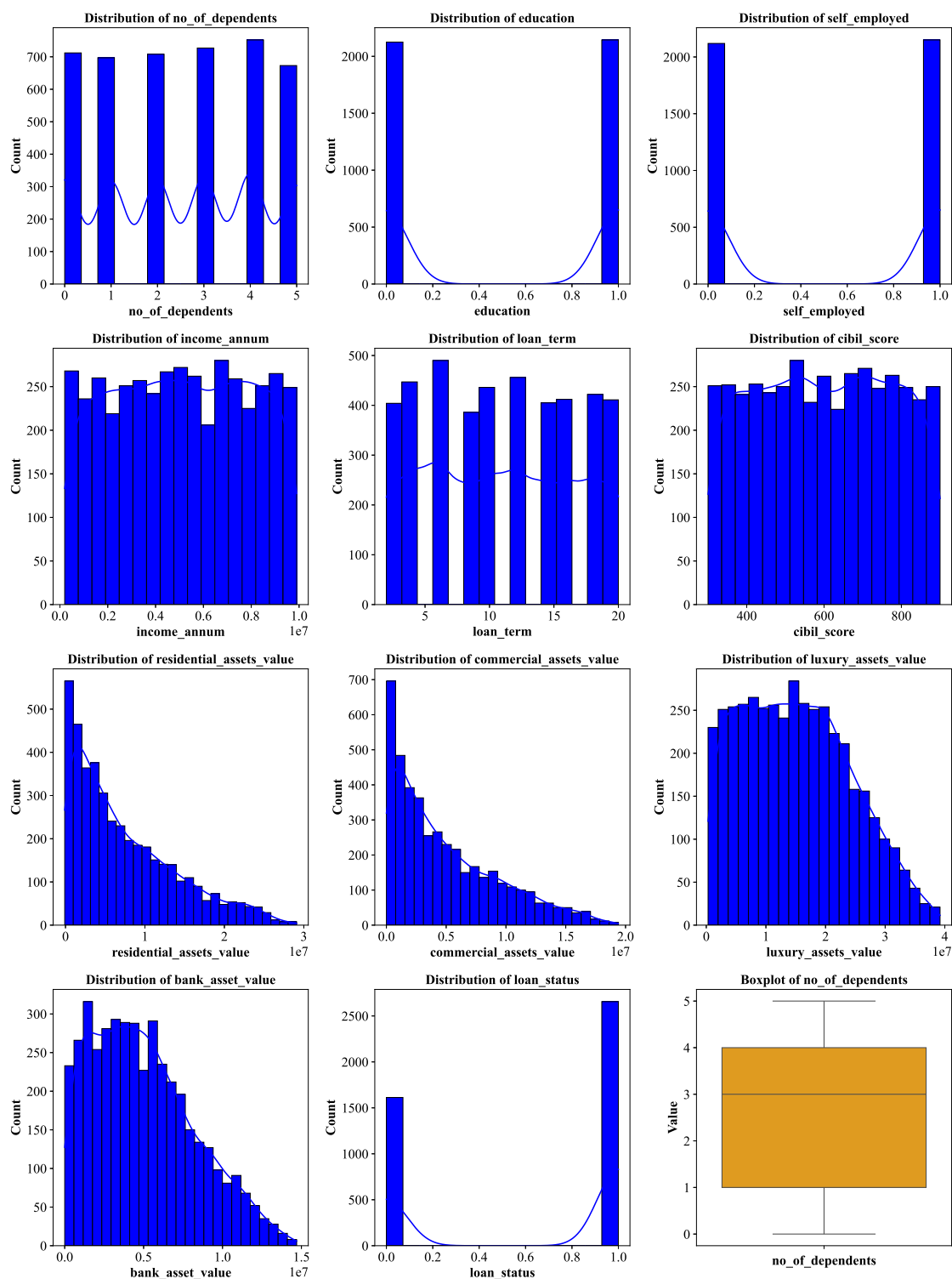


Figure 1: Consolidated EDA summary showcasing feature distributions prior to scaling.

Inference: ApplicantIncome and CoapplicantIncome are both pretty heavily skewed here, and that skew carries straight through to LoanAmount, which makes sense since the loan amount is presumably tied to how much someone earns.

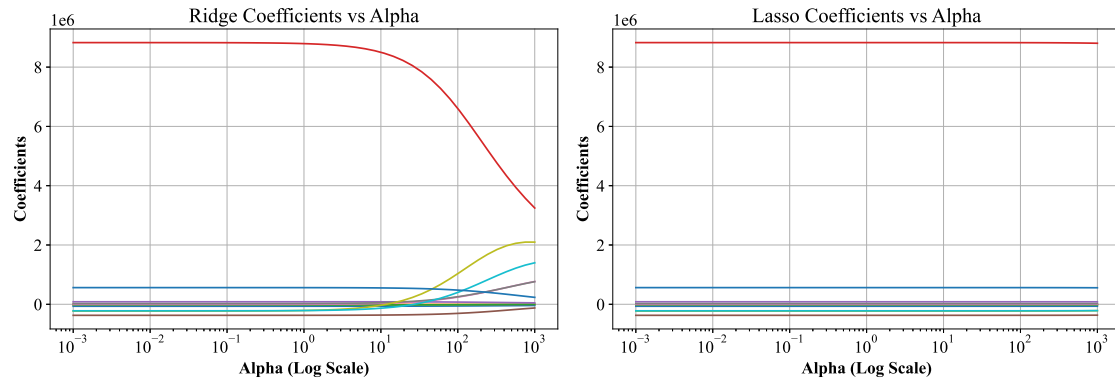


Figure 2: Comparison of model coefficients indicating shrinkage behavior across regularization methods.

Inference: This is basically what the theory predicts: OLS lets a few features carry a lot of weight, Ridge pulls all of the coefficients in a bit without zeroing anything out, and Lasso is more aggressive – several of the weaker coefficients get knocked down to exactly zero, leaving only the features that actually seem to matter.

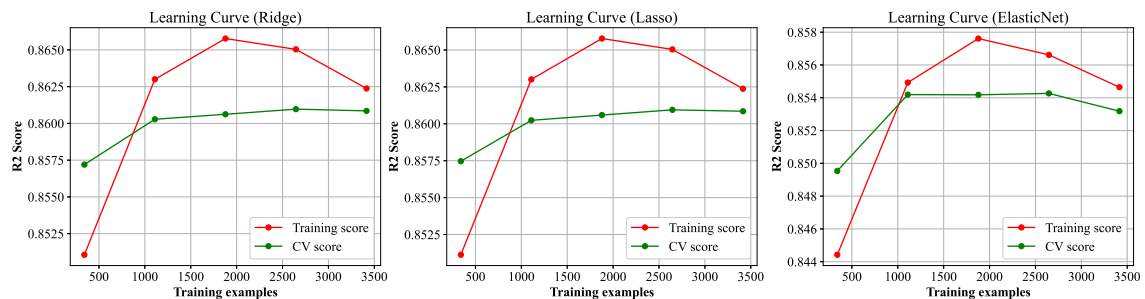


Figure 3: Learning curves for Ridge Regression indicating model stability across training sizes.

Inference: The training and validation curves come together as more data is added, so overfitting doesn't seem to be a major issue. The error stays fairly high overall though, which points more toward the linear model itself not being expressive enough to capture everything going on in this dataset, rather than a variance problem.

11 Discussion

Ridge edged out plain OLS slightly on MSE, which lines up with the general idea that a bit of bias in exchange for lower variance can help. ElasticNet didn't fare as well – its R^2 dropped noticeably, which suggests the $\alpha = 1.0$ setting used here was pushing the penalty a little too hard and the model ended up underfitting instead. The coefficient plot was probably the most useful part of this experiment for actually seeing Lasso's feature-selection behavior rather than just reading about it.

12 Conclusion

Between OLS, Ridge, Lasso, and ElasticNet, this experiment gave a reasonably concrete sense of the bias-variance tradeoff in action rather than just as an abstract concept. All plots were generated through `ml_utils.py` and kept in EPS format per the lab's formatting requirements.

13 References

1. Hoerl, A. E., and Kennard, R. W. "Ridge regression: Biased estimation for nonorthogonal problems." *Technometrics*, 1970.
2. Tibshirani, R. "Regression shrinkage and selection via the lasso." *Journal of the Royal Statistical Society: Series B (Methodological)*, 1996.